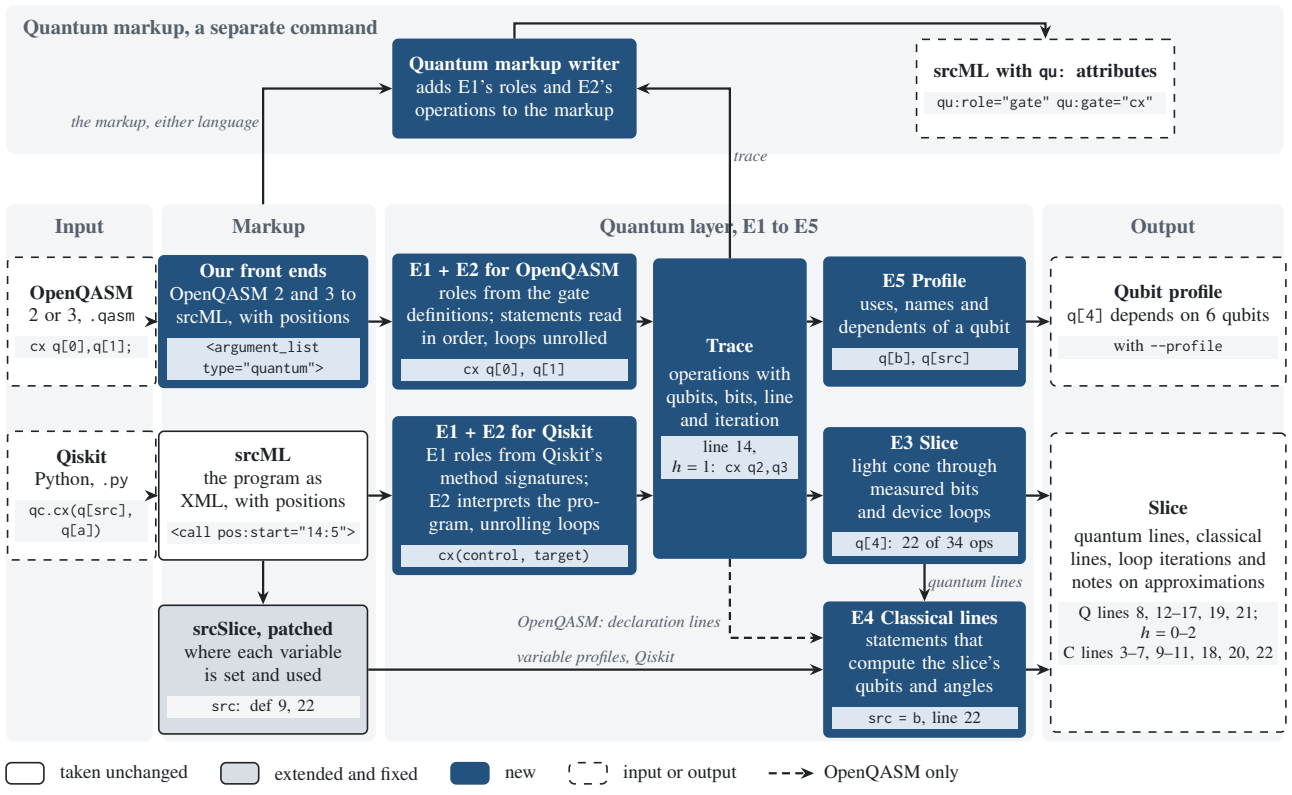

Quantum markup, a separate command
Quantum markup writer
adds E1's roles and E2's operations to the markup
srcML with qu: attributes
qu:role="gate" qu:gate="cx"
the markup, either language
trace
Input
Markup
Quantum layer, E1 to E5
Output
OpenQASM
2 or 3, .qasm
cx q[0],q[1];
Our front ends
OpenQASM 2 and 3 to srcML, with positions
<argument_list type="quantum">
E1 + E2 for OpenQASM
roles from the gate definitions; statements read in order, loops unrolled
cx q[0], q[1]
Trace
operations with qubits, bits, line and iteration
line 14, h = 1: cx q2,q3
E5 Profile
uses, names and dependents of a qubit
q[b], q[src]
Qubit profile
q[4] depends on 6 qubits
with --profile
Qiskit
Python, .py
qc.cx(q[src], q[a])
srcML
the program as XML, with positions
<call pos:start="14:5">
E1 + E2 for Qiskit
E1 roles from Qiskit's method signatures; E2 interprets the program, unrolling loops
cx(control, target)
E3 Slice
light cone through measured bits and device loops
q[4]: 22 of 34 ops
quantum lines
Slice
quantum lines, classical lines, loop iterations and notes on approximations
Q lines 8, 12–17, 19, 21; h = 0–2
C lines 3–7, 9–11, 18, 20, 22
srcSlice, patched
where each variable is set and used
src: def 9, 22
OpenQASM: declaration lines
variable profiles, Qiskit
E4 Classical lines
statements that compute the slice's qubits and angles
src = b, line 22
taken unchanged
extended and fixed
new
input or output
OpenQASM only